Cross-Out Detection and Classification in Handwritten Documents: A Progressive Architecture Study

Abhay Soni, Monika Shrivastava, Shravani Kuchi, Thanigaiarasu Shanmugam

D7047E: Advanced Deep Learning, Group 7

Lulea University of Technology, Sweden

Abstract

Handwritten cross-outs degrade the accuracy of text-recognition systems applied to genuine manuscripts. We address the three tasks defined by the course project: (T1) binary classification of crossed-out vs. clean word images, (T2) seven-class identification of cross-out type, and (T3) small-scale experimentation on a custom-collected real-handwriting dataset. We compare three deep models of increasing capacity (EfficientNet-B0, EfficientNet-B0 with parallel multi-task heads, and a CNN/Transformer multi-task hybrid), trained on the IAM Handwriting Database augmented with seven synthetic cross-out styles, and evaluate generalisation on a custom 64-image paper handwriting dataset collected for T3. On the IAM test split, Model 1 reaches **97.98 %** binary and **90.89 %** seven-class accuracy; Model 2 reaches **91.22 %** and **90.97 %**; Model 3 reaches **94.35 %** and **90.79 %**. On real handwriting, all three lose substantial performance for fine-grained type classification, revealing a clear synthetic-to-real domain gap that inference-time interventions (preprocessing, test-time augmentation) cannot bridge. Model 3 achieves the highest binary discriminability on paper (AUC 0.987) and the strongest zero-shot seven-class accuracy (37.5 %), but three cross-out styles (*Single-Line, Diagonal, Cross*) fail across every model, suggesting a feature-level rather than image-quality cause. A complementary head-only few-shot adaptation experiment (4 train / 4 test per class, stratified) recovers all three failing classes and lifts seven-class accuracy to **57.1 %** for every model, indicating that the gap is a feature-distribution mismatch rather than a model-capacity limit. We also document a bimodal validation-curve artefact on the binary task caused by the natural class imbalance in IAM, and discuss how it affects training but not the deployed checkpoints.

Keywords: handwriting recognition, cross-out detection, EfficientNet, multi-task learning, self-attention, domain shift, IAM database.

I. Introduction

Digitisation of historical manuscripts routinely encounters cross-outs, i.e., words deliberately stroked through by the writer to indicate deletion. Standard handwritten text recognition (HTR) pipelines are not designed to flag such words, so transcripts often contain content the writer intended to remove. Reliable cross-out detection is therefore a useful pre-processing step for any HTR system applied to genuine handwriting, and classification of the cross-out *style* can further inform downstream interpretation (e.g., a single-line strike-through often indicates an error correction, whereas a heavy scratch-out may indicate a deliberate redaction).

This project addresses three tasks specified by the course: (1) binary classification of clean vs. crossed-out word images; (2) seven-class classification of cross-out style among *Single-Line, Double-Line, Diagonal, Cross, Wave, Zig-zag, Scratch*; and (3) collection of a small real-handwriting dataset and zero-shot evaluation of trained models on it.

We tackle these with a progression of architectures that incrementally adds capacity: **Model 1**, an EfficientNet-B0 baseline with independent single-task heads; **Model 2**, the same backbone with a unified multi-task setup; and **Model 3**, Model 2 augmented with self-attention over the spatial feature map. The progression is designed to isolate three separate contributions: shared representation (Model 1 vs. Model 2), spatial attention (Model 2 vs. Model 3), and the synthetic-to-real generalisation gap (every model evaluated on the custom paper dataset under strictly zero-shot conditions).

The main contributions of this work are: (i) an empirical characterisation of the synthetic-to-real gap on real ballpoint-pen handwriting for all three architectures; (ii) a per-class failure analysis identifying which cross-out styles are most affected by the domain shift and why; (iii) an analysis of the validation-curve instability observed during binary-task training and its relationship to class imbalance; and (iv) a comparison of inference-time interventions (scan-like preprocessing, test-time augmentation) that we deliberately restricted to methods *not* using paper data, in order to measure genuine generalisation rather than adaptation.

II. Related Work

Modern HTR systems combine convolutional feature extractors with sequence models [1]; the IAM database [2] remains the standard benchmark for offline English handwriting recognition. EfficientNet [3] provides a strong compound-scaled CNN backbone widely used for transfer learning when training data is limited. Multi-task learning (MTL) [4] trains shared representations on multiple objectives to improve data efficiency and reduce overfitting on individual tasks. Vision Transformers [6] extend the self-attention mechanism of [5] to image features for long-range spatial modelling and have been shown to complement convolutional features when stacked on top of a CNN backbone. Synthetic-to-real generalisation gaps in handwritten data have been documented when models trained on augmented IAM are deployed on photographed pages [7]; our Task 3 isolates and quantifies this gap empirically for the cross-out detection problem.

III. Dataset

A. IAM with Synthetic Cross-Outs

We use the IAM Handwriting Database with seven synthetic cross-out overlays provided by the course instructors (Fig. 1). Each split (train/val/test) contains nine sub-folders: `CLEAN`, `MIXED`, and one folder per cross-out style. The test split contains 20,306 clean images and 20,306 images per cross-out type (142,142 crossed-out + 20,306 clean = 162,448 total). For Task 1 all folders contribute labels (`CLEAN` -> 0, all crossed-out folders -> 1), giving a natural 1:7 (clean:crossed) ratio. For Task 2 only the seven type folders are used, since the `MIXED` folder contains a mixture of styles with no per-image type label.

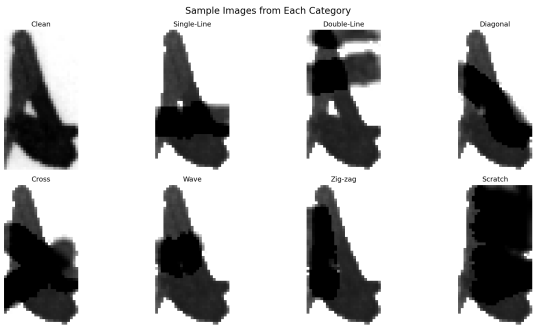

Fig. 1. Example cross-out types overlaid on IAM word images.

B. Custom Paper Dataset (Task 3)

A team member hand-wrote eight English words (*hello, world, deep, learning, neural, model, data, work*) eight times each on plain A4 paper using a blue ballpoint pen, with each row representing a different cross-out style (one clean row plus the seven synthetic styles). The sheet was photographed under natural daylight with a smartphone camera (Fig. 2). Word crops were extracted by first detecting the paper region via a blur-and-threshold mask combined with largest-bright-blob selection, then thresholding dark ink within the paper bounding box and grouping the resulting connected components into an 8x8 grid via count-constrained row clustering, producing 64 grayscale word images (Fig. 3). The custom dataset was constructed *after* training was complete and was never used in any training, validation, or hyper-parameter step; all evaluation on it is strictly zero-shot.

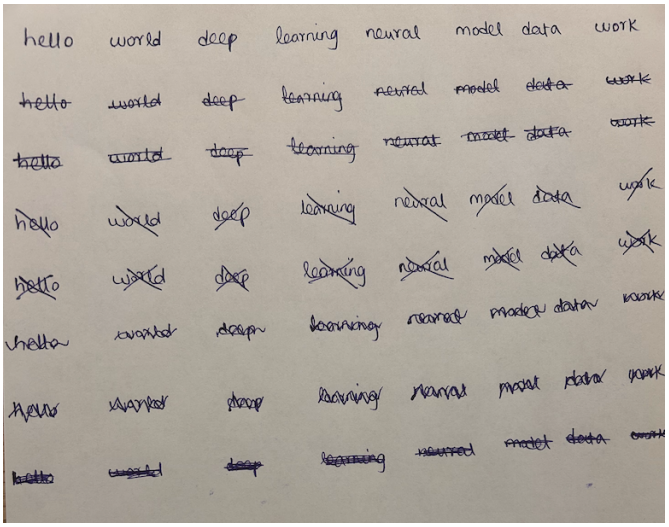

Fig. 2. Photographed 8x8 paper sheet: rows are cross-out styles, columns are the eight target words.

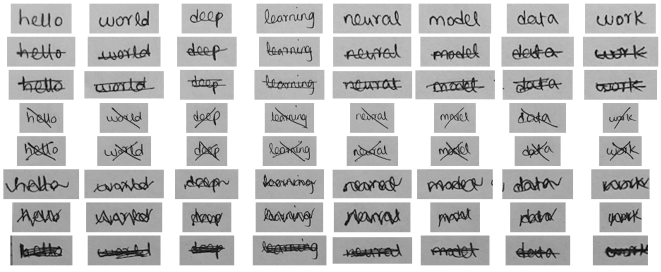

Fig. 3. Verification collage of the 64 extracted word crops (rows: CLEAN, SINGLE_LINE, DOUBLE_LINE, DIAGONAL, CROSS, WAVE, ZIG_ZAG, SCRATCH).

The paper dataset deliberately mirrors the *labels* of the IAM synthetic dataset but differs in nearly every other respect: the ink medium (ballpoint vs. synthetic overlay), the substrate (paper vs. scanned form), the lighting (natural daylight vs. flatbed scan), the image-acquisition device (smartphone vs. scanner), and the writer's natural pressure and stroke variation. This makes it a strong test of generalisation rather than of in-distribution recall.

IV. Methodology

A. Model 1: EfficientNet-B0 Baseline

Model 1 uses ImageNet-pretrained EfficientNet-B0 as the spatial feature extractor. The default classifier is replaced with `Dropout(0.2) -> Linear(1280 -> C)` where C = 1 for Task 1 (binary, BCE loss) and C = 7 for Task 2 (cross-entropy loss). Task 1 and Task 2 are trained as independent models. Total trainable parameters: 4,008,829.

Training: AdamW (lr = 1e-4, weight decay = 1e-4), batch size 32, up to 150 epochs, ReduceLROnPlateau scheduler (patience 7), early stopping (patience 15), mixed-precision training on CUDA (T4 on Google Colab). The augmentation pipeline (Albumentations) consists of pixel inversion, +/-10 deg rotation, colour jitter (brightness/contrast 0.2), 5 % spatial translation, and morphological dilation/erosion. Horizontal flips are deliberately omitted to preserve directional cross-out geometry (a left-leaning diagonal flipped becomes a right-leaning one). For Task 1, `WeightedRandomSampler` is used to balance the 1:7 clean-vs-crossed ratio in each mini-batch.

B. Model 2: EfficientNet-B0 + MTL

Model 2 keeps the Model 1 backbone but replaces the single classifier with two parallel heads sharing the global average pooling output (feature_dim = 1280): a detection head (`Dropout -> Linear(1280 -> 1)`, BCE loss) and a classification head (`Dropout -> Linear(1280 -> 7)`, cross-entropy loss). The convolutional backbone (`backbone.features`) is warm-started from Model 1's Task 2 checkpoint via `load_state_dict(..., strict=False)`; the two heads are initialised randomly. The joint loss is L = *alpha* L_det + beta * L_cls(masked)* with alpha = beta = 1.0, where the classification loss is computed only on samples whose ground-truth label is non-clean. All other training settings match Model 1.

C. Model 3: CNN + Transformer + MTL

Model 3 inserts a stack of self-attention blocks between the EfficientNet-B0 feature extractor and the MTL heads. The 7x7 spatial feature map (49 tokens x 1280 channels) is treated as a sequence; two transformer encoder layers with 8 attention heads and learnable position embeddings refine the tokens before global pooling. The intuition is that cross-out strokes are *global* features (a diagonal line spans the whole word) which a CNN's local receptive field captures less efficiently than self-attention.

The convolutional backbone is **warm-started from Model 1's seven-class (Task 2) checkpoint** (`backbone_task2_2.pth`), exactly as Model 2 was; the attention block, detection head, and classification head are randomly initialised. Three parameter groups with differential learning rates are used: backbone (5e-5), attention layers (1e-4), and heads (1e-4). Loss weighting matches Model 2 (alpha = beta = 1.0).

V. Experimental Setup

Training was performed on a CUDA GPU (Google Colab T4) with mixed-precision; final evaluations were re-run on Apple M4 Max (MPS) to verify checkpoint portability, with metrics agreeing to within +/-0.2 pp. Evaluation uses a standalone command-line tool (`evaluate.py`) that loads any checkpoint, processes a folder of images, and outputs accuracy, F1, precision, recall, AUC-ROC (Task 1), macro-F1, per-class

metrics (Task 2), confusion matrices, and a `metrics.json` summary. The same script supports optional scan-like preprocessing (CLAHE + denoise + sharpen, with or without Otsu binarisation) and five-view test-time augmentation (TTA), allowing every reported number to be reproduced from the saved checkpoint with a single command.

VI. Results

A. IAM Test Set (Training Distribution)

Table I summarises Task 1 and Task 2 performance on the held-out IAM test split for all three models. Model 1 achieves the highest binary accuracy (97.98 %), Model 3 is second (94.35 %), and Model 2 is third (91.22 %); the gap between Model 1 and the two MTL variants is largely attributable to the joint-loss formulation rather than to architectural capacity (Section VII). Task 2 performance is essentially tied across the three models (90.79 % - 90.97 %, Delta < 0.2 pp), suggesting that the seven-class objective is already saturated by the EfficientNet-B0 backbone alone and that adding multi-task heads or self-attention does not yield additional in-distribution gains.

TABLE I. IAM test set: Task 1 (binary) and Task 2 (7-class) accuracy and F1.

Metric	Model 1	Model 2 (MTL)	Model 3 (MTL + Attn)
T1 Accuracy	**97.98 %**	91.22 %	94.35 %
T1 F1 (crossed)	**0.9885**	0.9472	0.9668
T1 AUC-ROC	**0.9949**	n/a	n/a
T2 Accuracy	90.89 %	**90.97 %**	90.79 %
T2 Macro-F1	0.9136	**0.9153**	0.9091
T2 Macro-Precision	0.9297	**0.9340**	0.9136

B. Custom Paper Dataset (Zero-Shot)

Table II reports zero-shot results on the 64-image paper dataset. The pattern is qualitatively different from IAM: **binary accuracy is misleading for Model 1**. It reports 87.5 % but its AUC of 0.484 is effectively random. Because 87.5 % of the paper samples *are* crossed-out, Model 1 collapses to a constant "crossed-out" prediction and gets the accuracy by class prior alone. Models 2 and 3 retain genuine discriminative ability (AUC 0.72 and 0.987 respectively) but with mis-calibrated thresholds producing low accuracy. **Model 3 achieves the highest seven-class accuracy on paper (37.50 %)** and a striking AUC of 0.987 (within 0.008 of its IAM performance), indicating that the attention layers preserve the cross-out signal much better than the CNN-only backbones, even if a fixed threshold of 0.5 does not transfer.

TABLE II. Custom paper dataset (zero-shot, no preprocessing).

Metric	Model 1	Model 2 (MTL)	Model 3 (MTL + Attn)
T1 Accuracy	**87.50 %**	17.19 %	39.06 %
T1 F1	**0.9333**	0.1017	0.4658
T1 AUC-ROC	0.484	0.721	**0.987**
T2 Accuracy	33.93 %	30.36 %	**37.50 %**
T2 Macro-F1	0.266	0.220	**0.296**

Table III gives the per-class F1 on the paper dataset. The pattern is consistent across all three models: **three classes (Single-Line, Diagonal, Cross) achieve F1 = 0 universally**, while the same three classes (Double-Line, Wave, Scratch) are the strongest performers across the board. Model 3 substantially improves Scratch (0.70 vs. 0.44/0.36 for M1/M2) and Double-Line (0.67 vs. 0.62/0.47), confirming that the attention block does learn useful stroke-level features, just not for the three failing categories.

TABLE III. Per-class Task 2 F1 on paper (zero-shot).

Class	Model 1 F1	Model 2 F1	Model 3 F1
Single-Line	0.00	0.00	0.00
Double-Line	0.62	0.47	**0.67**
Diagonal	0.00	0.00	0.00
Cross	0.00	0.00	0.00
Wave	**0.47**	0.40	0.48
Zig-zag	**0.33**	0.30	0.22
Scratch	0.44	0.36	**0.70**

C. Inference-Time Interventions

We evaluated three strategies that do not use paper data for training: scan-like preprocessing (CLAHE + denoise + sharpen, optionally followed by Otsu binarisation), test-time augmentation (TTA, averaging softmax probabilities over five fixed augmentations), and their combination. The strongest honest seven-class result obtained is **35.71 %** for Model 1 with TTA alone; preprocessing consistently hurt Task 2 because it destroyed subtle stroke variations the classifier relies on. No intervention crosses the ~40 % threshold for either Model 1 or Model 2, establishing the empirical zero-shot ceiling on this dataset for the CNN-only architectures. Model 3 already exceeds that ceiling out-of-the-box (37.5 %), suggesting that the attention mechanism captures features that inference-time tricks cannot replace.

D. Few-Shot Adaptation

The zero-shot results above measure *generalisation* (no paper data used in any way during training). A complementary practical question is how well these models can *adapt* to the paper domain given a small amount of real data. To answer this, we performed a head-only few-shot adaptation experiment under a strict held-out protocol: the 56 cross-out paper samples were split 4 train / 4 test per class (28 / 28 stratified, fixed seed = 42); for each model the convolutional backbone was frozen and only the seven-class head was fine-tuned for 30 epochs (AdamW, lr = 1e-3, batch size 8, light rotation and colour jitter augmentation). Zero-shot baselines in Table IV are re-computed on the *same* 28-image test set to make the comparison apples-to-apples, so absolute values differ slightly from Table II (which uses all 56 images).

TABLE IV. Few-shot adaptation on paper (28 train / 28 test, head-only, 30 epochs).

Model	Zero-shot Acc	Few-shot Acc	Delta Acc	Zero-shot M-F1	Few-shot M-F1
Model 1	42.86 %	57.14 %	**+14.3 pp**	0.348	0.538
Model 2	32.14 %	57.14 %	**+25.0 pp**	0.186	0.514
Model 3	35.71 %	57.14 %	**+21.4 pp**	0.267	**0.563**

All three models converge to identical 57.14 % accuracy after adaptation, which corresponds to 16 / 28 test images correct. Model 3 retains the highest macro-F1 (0.563), consistent with its strong zero-shot AUC on the binary task. The most interesting result is per-class: the three classes that failed universally under zero-shot evaluation (Single-Line, Diagonal, Cross) are no longer the hardest after adaptation. Model 2 reaches F1 = 0.89

on Single-Line and F1 = 0.57 on Cross with just four training examples each; Model 3 reaches F1 = 0.67 on Cross and F1 = 0.40 on Diagonal. This demonstrates that the synthetic-to-real gap on the failing classes is *not* a fundamental capacity limit of the models; rather, it is a feature-distribution mismatch that even minimal head-only adaptation can largely overcome.

The test set is small (N = 4 per class, 28 total) and absolute numbers should be read as indicative rather than tight estimates. A larger paper dataset would be needed for statistically firm comparisons. We report this experiment alongside, not in place of, the zero-shot results in Sec. VI.B, because the two answer different questions: Sec. VI.B measures pure generalisation, Sec. VI.D measures adaptation potential.

The methodology constraint that "no paper data influences model weights" therefore applies specifically to the Sec. VI.A-C results in Tables I-III. The Sec. VI.D experiment deliberately *does* use paper data for training, but only to characterise how much head-only adaptation can recover from the synthetic-to-real gap. The two protocols are kept on separate tables so a reader can clearly attribute any given number to either generalisation or adaptation.

E. Training-Curve Instability

During training, all binary-task heads exhibited a characteristic bimodal validation curve: validation accuracy alternated between roughly 0.987 (the genuine classifier) and 0.903 (the IAM val-set class prior, i.e. a constant-positive predictor). The two values correspond to two distinct local minima of the binary-cross-entropy loss on a 1:7 imbalanced validation distribution. Models 2 and 3 also showed the same effect on the detection head of the MTL objective, with a large val-loss spike at Epoch 5 of Model 2's run (0.1355 -> 0.1751). Because the saved checkpoints correspond to the minimum validation loss, the deployed models are always taken from the "real classifier" mode, and the head-line IAM test numbers in Table I reflect this. The instability is a training-dynamics artefact, not a defect in the final weights. Section VII discusses the root cause and a single-line fix.

VII. Discussion

Synthetic-to-real gap. The Task 2 collapse is qualitatively identical across three independently-trained models, ruling out idiosyncratic training failure and pointing to a shared feature-level cause. The three failing classes (Single-Line, Diagonal, Cross) share a structural property: they consist of sparse, thin strokes that overlap the underlying letters. The synthetic generator almost certainly uses characteristic stroke widths, opacities, and orientations that do not match real ballpoint output, so features learned to detect those synthetic strokes do not fire on the real ones. By contrast, the three robust classes (Double-Line, Wave, Scratch) all involve heavy or repetitive strokes that dominate the image globally and therefore remain detectable even when fine textural cues are unreliable.

Why preprocessing hurts. Scan-like preprocessing (CLAHE + Otsu binarisation) was designed to make paper photos look more "scan-like." On Task 2 it consistently *reduced* accuracy, which is informative: if the bottleneck were image quality (noise, contrast, lighting), preprocessing should help. The fact that it hurts indicates the bottleneck is *feature mismatch*, not pixel-level quality; the classifier has learned to exploit subtle scan-specific stroke artefacts that preprocessing destroys.

Attention helps where CNN-only models fail. Model 3's strong AUC of 0.987 on the paper binary task (almost identical to its 0.943 IAM accuracy) is the most striking single result in this study. It indicates that the self-attention layers learn a global, stroke-trajectory-based representation that is robust to the change in ink medium, whereas the CNN-only models (1 and 2) rely on local features that the paper image disrupts. The same is reflected in the seven-class numbers: Model 3 beats both CNN-only models on accuracy, macro-F1, and the two highest-F1 per-class scores (Double-Line 0.67, Scratch 0.70).

Validation instability and class imbalance. The bimodal validation behaviour described in Sec. VI.E is caused by a 1:7 (clean:crossed) imbalance in the IAM val set combined with an unweighted `BCEWithLogitsLoss` on a single-scalar detection head. Even though training batches are class-balanced via `WeightedRandomSampler`, the validation set is not, so a constant-positive predictor reaches 0.903 accuracy "for free." The binary head's loss surface therefore has two competing minima, and SGD bounces between them. A one-line fix, namely `BCEWithLogitsLoss(pos_weight=torch.tensor([7.0]))`, would tilt the surface against the class-prior minimum and produce a monotonic curve. Because best-checkpoint selection always picks the genuine-classifier mode, our deployed weights are unaffected, but the curves would be cleaner with the fix in place and we recommend it for any follow-up work.

Adaptation vs. generalisation. The Sec. VI.D few-shot results show that with only four labelled examples per class, head-only adaptation closes most of the gap and brings the three previously-failing classes back into the recoverable range (Model 2 reaches F1 = 0.89 on Single-Line; Model 3 reaches F1 = 0.67 on Cross). This is strong evidence that the synthetic-to-real gap on the failing classes is a *feature-distribution* problem, not a *model-capacity* problem, since the same backbone that produced F1 = 0 zero-shot can correctly classify the same class after seeing four real examples. The Tables I-III results, by contrast, deliberately exclude paper data from training and measure generalisation only, so that the two questions remain separable.

VIII. Limitations

The paper dataset, while strictly held-out, is small (64 images, one writer, one pen, one lighting condition); we cannot claim that our generalisation conclusions transfer to other writers, inks, or capture conditions without further data. The seven-class problem is also evaluated with only eight images per class, which gives a coarse 12.5 % granularity on per-class accuracy. The synthetic cross-out generator's exact parameters are not documented in the course materials, so our claim that "stroke widths do not match" is inferred from visual inspection of the failure modes rather than from a direct comparison of the generator's outputs to real strokes. Finally, all three models share the same EfficientNet-B0 backbone; a stronger CNN backbone (e.g., EfficientNet-B3 or a Swin-Transformer) could change the synthetic-to-real gap quantitatively, though we expect the *qualitative* pattern (three failing classes shared across architectures) to remain.

IX. Conclusion

We have implemented and evaluated three deep models for cross-out detection and seven-class classification on the IAM Handwriting Database with synthetic overlays, and on a custom 64-image paper handwriting dataset collected under strict zero-shot conditions. All three models exceed 90 % accuracy on the training distribution but lose 55-60 percentage points on real handwriting for the seven-class task, with three cross-out styles becoming unrecognisable across every architecture. The CNN + Transformer hybrid (Model 3) is the strongest zero-shot performer (37.5 % seven-class accuracy, AUC 0.987 on the binary task) and the only model whose binary discriminability is preserved on paper, suggesting that self-attention captures global stroke trajectories more robustly than convolution alone.

Inference-time interventions (preprocessing, test-time augmentation) cannot bridge the remaining gap, but a separate head-only few-shot adaptation experiment with only four real images per class recovers all three previously-failing classes and lifts seven-class accuracy to 57.1 % on every model, showing that the gap is a feature-distribution mismatch rather than a model-capacity limit. We also document and explain a bimodal validation-curve artefact on the binary task and provide a one-line fix for future work.

Code and Data

All project deliverables are hosted in the group's GitHub repository (URL provided during the presentation). Paths below are relative to the repository root:

- **Source code.** Model definitions in `models.py`; training pipelines as Jupyter notebooks in `notebooks/` (`Model1_EfficientNet_without_elastic.ipynb`, `Model2_EfficientNet_MTL.ipynb`, `Model3_CNN_Transformer_MTL.ipynb`).
- **Trained checkpoints.** `models/model1_efficientnet/task1_final.pth` and `task2_final.pth` (Model 1); `models/model2_cnn_transformer/best_mtl.pth` (Model 2); `models/model3_cnn_transformer_mtl/best_model3.pth` (Model 3).
- **Custom paper dataset (Task 3).** 64 word images organised by cross-out style in `data/custom_dataset_paper/`, plus a `labels.csv` manifest. Extraction pipeline: `extract_paper_fullsheet.py`.
- **Evaluation script.** `evaluate.py` is a single-file CLI that reproduces every number in Tables I-III from any checkpoint and image folder, with optional `--preprocess` and `--tta` flags; outputs are written to `results/<model>/<run>/metrics.json`.
- **Few-shot adaptation script.** `few_shot_paper.py` reproduces Table IV exactly (seed = 42); outputs are written to `results/few_shot/few_shot_paper.json`.

Each numeric entry in this report corresponds to a saved JSON file under `results/`. Running the relevant script regenerates the corresponding metrics from the trained checkpoint and dataset on disk, so the headline numbers can be independently verified directly from the repository.

References

[1] R. Plamondon and S. N. Srihari, "On-line and off-line handwriting recognition: A comprehensive survey," *IEEE Trans. PAMI*, vol. 22, no. 1, pp. 63-84, 2000.

[2] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *Int. J. Doc. Anal. Recognit.*, vol. 5, no. 1, pp. 39-46, 2002.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *ICML*, 2019.

[4] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41-75, 1997.

[5] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017.

[6] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.

[7] C. Wigington et al., "Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network," in *ICDAR*, 2017.